<table>
<tr><td colspan="2">Nombre:

Ángel Adrián Velez Contreras

Alonso Morales Quintanilla

Jairany López Ramos</td><td>Matrícula:

2145646

2145757

2145407</td></tr>
<tr><td colspan="2">Nombre del curso: Componentes en Lógica de Negocios</td><td>Nombre del profesor: Miguel Ángel Gómez Marroquín</td></tr>
<tr><td colspan="2">Unidad: 6</td><td>Actividad: PIA

Link del repositorio:

https://github.com/jyrnso0/PIA</td></tr>
<tr><td colspan="3">Fecha: 6/11/2026</td></tr>
<tr><td colspan="3">Bibliografia (Formato APA)

FastAPI. (s. f.). FastAPI. Recuperado el 6 de noviembre de 2026, de https://fastapi.tiangolo.com/

Pydantic. (s. f.). Pydantic. Recuperado el 6 de noviembre de 2026, de https://docs.pydantic.dev/latest/

SQLAlchemy. (s. f.). SQLAlchemy documentation. Recuperado el 6 de noviembre de 2026, de https://docs.sqlalchemy.org/en/20/

Docker. (s. f.). Docker documentation. Recuperado el 6 de noviembre de 2026, de https://docs.docker.com/

GitHub. (s. f.). GitHub Actions documentation. Recuperado el 6 de noviembre de 2026, de https://docs.github.com/en/actions</td></tr>
</table>

Objetivo:

Diseñar, desarrollar y validar AulaPlus como una plataforma de servicios de tutoría con una API REST que implemente sus reglas de negocio para gestionar reservas, pagos, sesiones, cancelaciones y disputas, aplicando una arquitectura basada en dominio y verificando su funcionamiento mediante pruebas automatizadas y trazabilidad entre los requerimientos y la solución.

1. Resumen ejecutivo

AulaPlus es una plataforma que conecta estudiantes con tutores particulares y cobra una comisión del 20 % por sesión. Este proyecto integra lo trabajado en las unidades anteriores —BRD, historias de usuario, glosario, BPMN y la arquitectura DDD de la Unidad 3— en una aplicación funcional: una API REST con DDD, Specification y State Machine, persistencia relacional, pruebas automatizadas, Docker Compose, CI/CD y OpenAPI. Las 20 reglas de negocio del BRD (BR-0001 a BR-0020) están implementadas y verificadas por pruebas.

Indicador	Resultado
Reglas del BRD implementadas	20 de 20
Historias de usuario con Gherkin	20 (en `docs/gherkin/`)
Rutas REST documentadas en OpenAPI	35
Pruebas automatizadas	101 (unitarias e integración)
Cobertura de código	99.25 % en la última ejecución contra PostgreSQL 16 (umbral exigido: 80 %)
Bases de datos probadas	SQLite y PostgreSQL 16

2. Análisis

2.1 Contexto, problemas y stakeholders

El negocio sufre disputas sobre si la sesión ocurrió (falta de evidencia), traslapes de agenda, cancelaciones sin política y reembolsos decididos por WhatsApp. Los stakeholders son Estudiante, Tutor, Plataforma/Administración y Soporte. El alcance cubre reserva con pago, disponibilidad, evidencia de sesión, cancelaciones, disputas, comisión y reembolsos, suspensión de tutores y dashboard. El BRD completo está en `docs/BRD.md`; incluye restricciones, supuestos, criterios de éxito y la tabla de 10 decisiones de refinamiento que precisan puntos ambiguos sin cambiar la empresa ni las reglas.

2.2 Reglas de negocio y historias

Regla	Descripción	Historia	Prioridad
BR-0001	Pago en 15 min; si no, se libera el horario	US-01	Alta
BR-0002	Cierre a las 48 h o cuando ambos califican	US-02	Media
BR-0003	Cancelan estudiante/tutor; Soporte solo con situación reportada	US-03	Alta

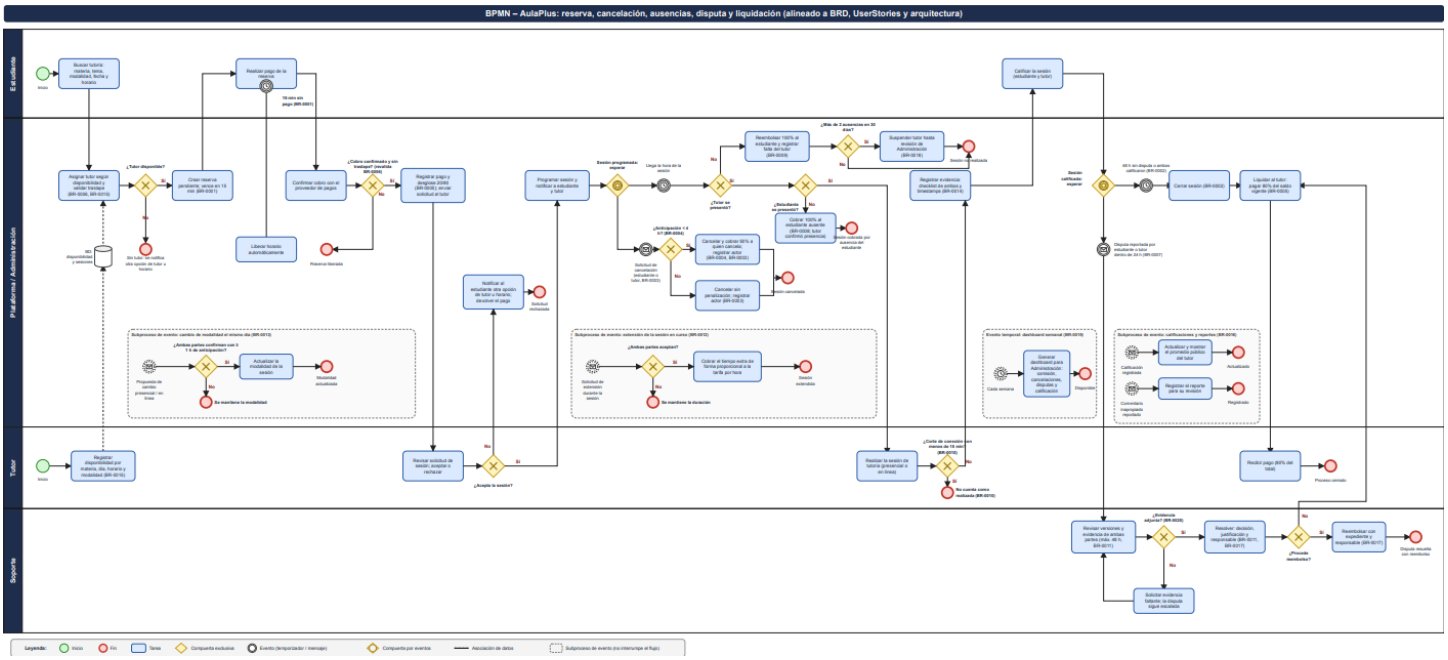
BR-0004	50 % de penalización con menos de 4 h	US-04	Alta
BR-0005	Comisión 20 % plataforma / 80 % tutor	US-05	Crítica
BR-0006	Sin traslapes para un mismo tutor	US-06	Alta
BR-0007	Disputa hasta 24 h después de la sesión	US-07	Alta
BR-0008	Cobro 100 % al estudiante ausente si el tutor confirmó presencia	US-08	Media
BR-0009	Reembolso 100 % por ausencia del tutor y registro de falta	US-09	Alta
BR-0010	Corte de conexión cuenta si duró ≥ 15 min	US-10	Media
BR-0011	Soporte resuelve disputas escaladas en ≤ 48 h	US-11	Crítica
BR-0012	Extensión con ambas partes y cargo proporcional	US-12	Baja
BR-0013	Cambio de modalidad con ambas partes y ≥ 1 h de anticipación	US-13	Baja
BR-0014	Checklist de ambas partes y timestamps de inicio/fin	US-14	Alta
BR-0015	Disponibilidad independiente por materia, día, horario y modalidad	US-15	Media
BR-0016	Promedio público y reporte de comentarios inapropiados	US-16	Baja

BR-0017	Reembolso con evidencia (motivo) y responsable	US-17	Alta
BR-0018	Suspensión con más de 2 ausencias en 30 días	US-18	Media
BR-0019	Dashboard semanal: comisión, cancelaciones, disputas, calificación	US-19	Media
BR-0020	No se cierra una disputa sin evidencia	US-20	Crítica

Cada historia tiene criterios de aceptación en Gherkin (Dado/Cuando/Entonces) en `docs/user_stories.md` y `docs/gherkin/*.feature`, y un glosario de 27 términos con su representación en el código (`docs/glossary.md`).

2.3 BPMN adaptado

Se conservaron los **cuatro carriles** (Estudiante, Plataforma/Administración, Tutor, Soporte) y el flujo base del BPMN original (seleccionar materia/tema/modalidad/fecha/horario → reserva → pago → validar → asignar tutor → ¿tutor disponible? → aceptar → sesión → asistencia → calificación; Soporte: ¿incidencia? → soporte → cerrar ticket).

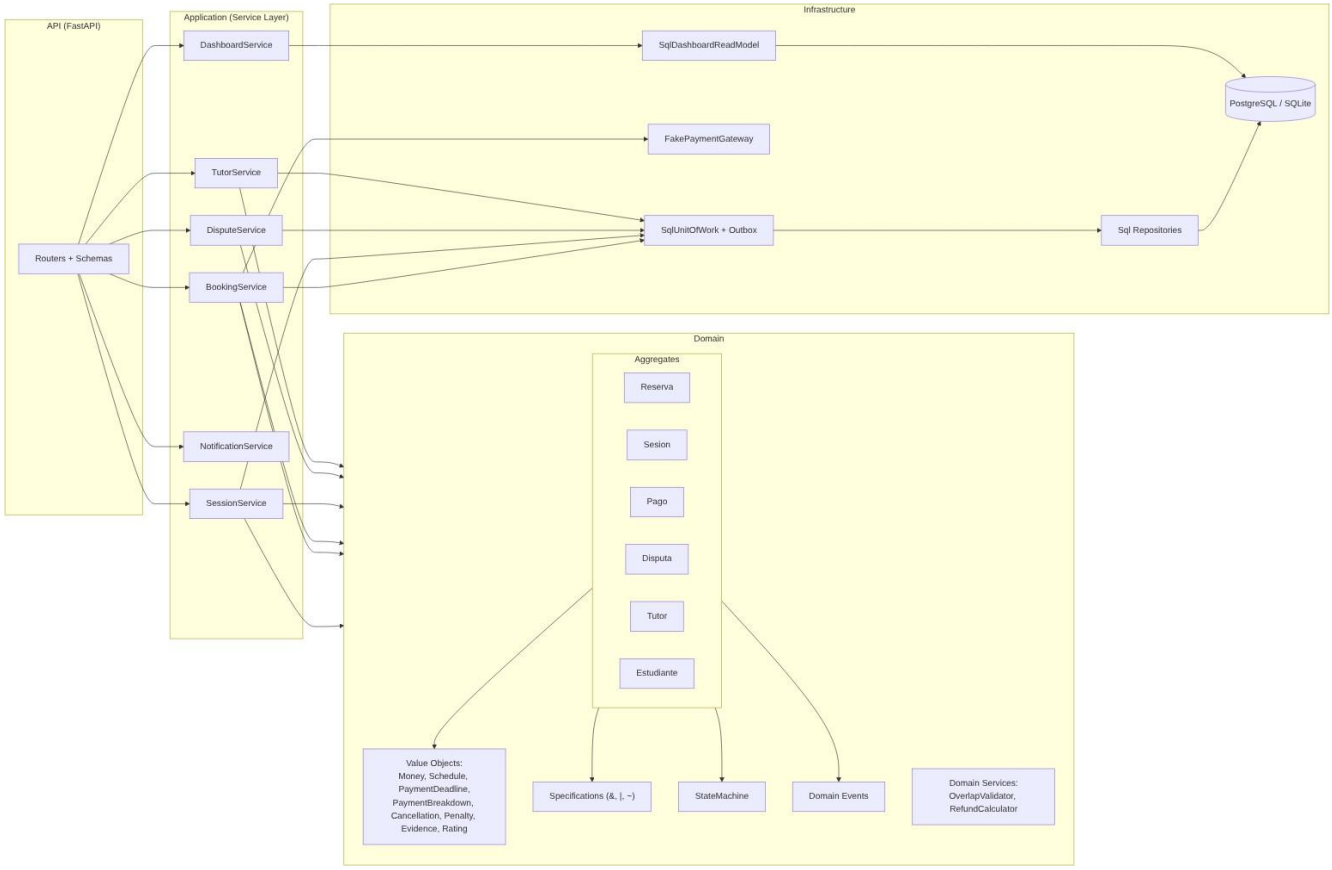


BPMN adaptado — los elementos amarillos son las reglas incorporadas

Elemento agregado	Regla
Compuerta «¿pago en ≤ 15 min?» y liberación del horario	BR-0001
Cancelación, compuerta «< 4 h» y penalización 50 %	BR-0003, BR-0004
Corte de conexión ≥ 15 min	BR-0010
Confirmación de presencia del tutor y ausencia del tutor con reembolso 100 %	BR-0008, BR-0009
Disputa ≤ 24 h, evidencia obligatoria, resolución en ≤ 48 h con responsable	BR-0007, BR-0011, BR-0017, BR-0020
Cierre con liquidación (ambos califican o 48 h)	BR-0002
Suspensión por ausencias y dashboard semanal	BR-0018, BR-0019

3. Arquitectura

3.1 Capas y aggregates (DDD)



Capas, aggregates y patrones

Monolito modular con arquitectura hexagonal: `src/api` → `src/application` → `src/domain` ← `src/infrastructure`.

Los aggregates (Reserva, Sesion, Pago, Disputa, Tutor) son las fronteras de consistencia; los Value Objects (`Money`, `Schedule`, `PaymentDeadline`, `Cancellation`, …) son inmutables; los servicios de dominio (`OverlapValidator`, `RefundCalculator`) alojan reglas que cruzan aggregates.

3.2 Modelo UML de clases

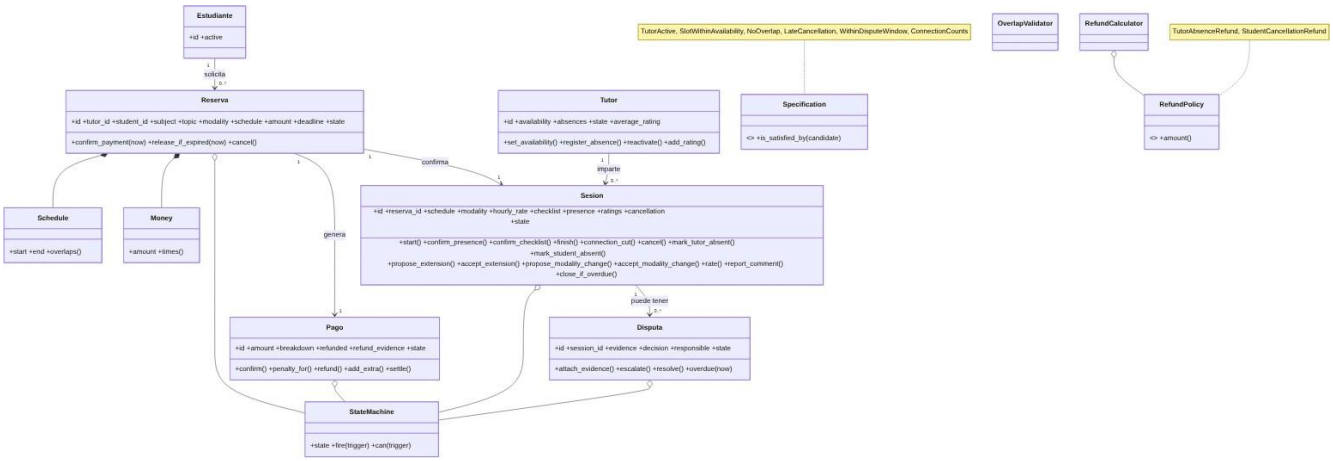
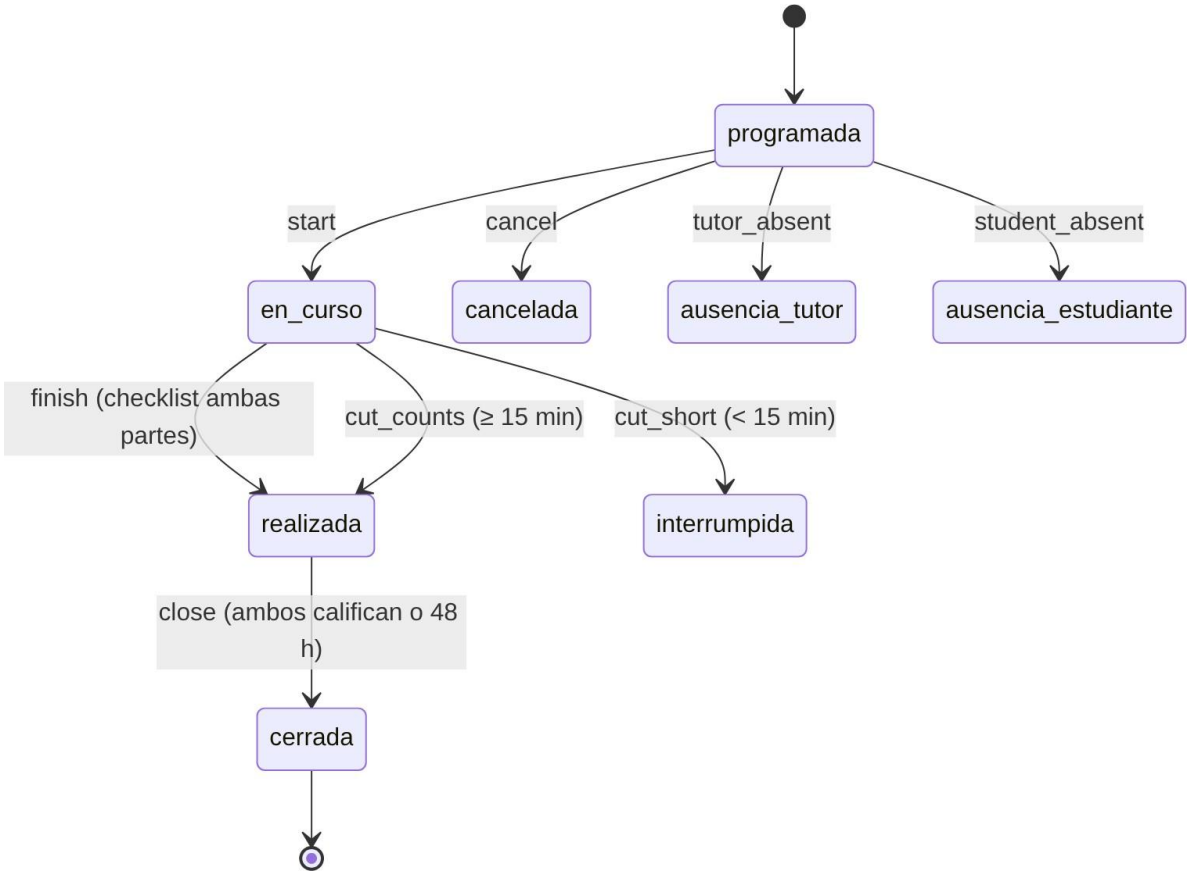


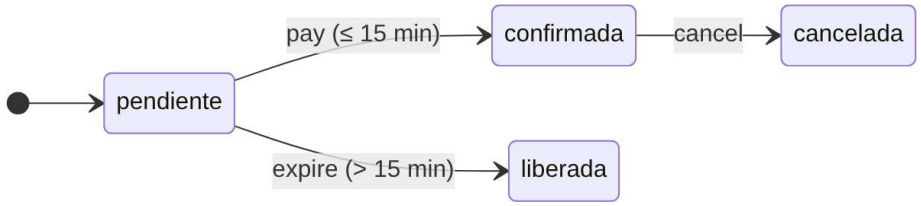
Diagrama de clases

3.3 State Machine

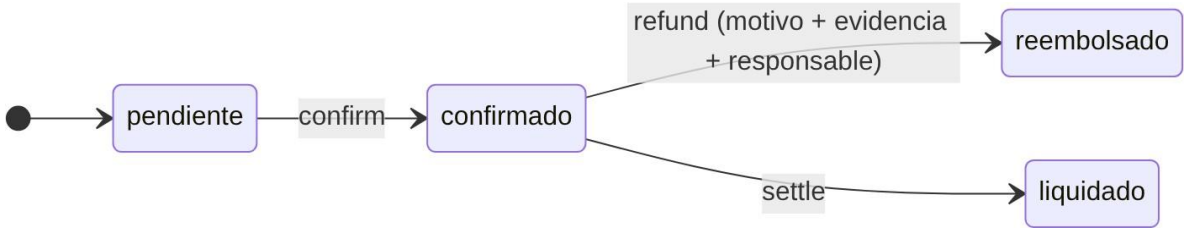
Reserva, Sesión, Pago y Disputa se gobiernan con una tabla de transiciones declarativa; cualquier salto inválido responde `409`.



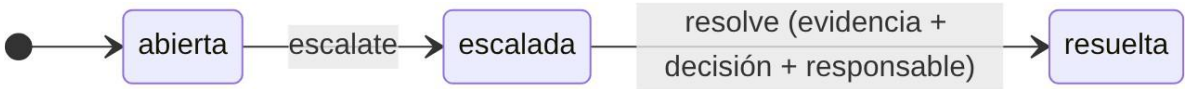
Estados de la Sesión



Estados de Reserva



Estados de Pago



Estados de Disputa

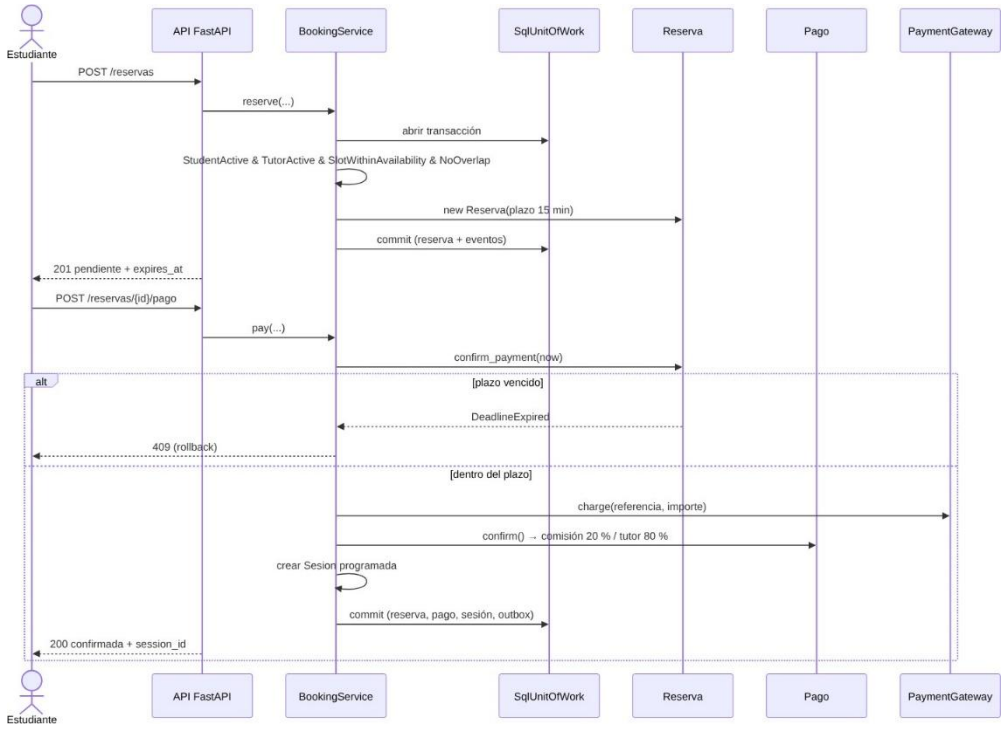
3.4 Patrones, SOLID y ADRs

Patrón	Dónde	Justificación
Specification (&, |, ~)	`domain/specifications.py`	Reglas de elegibilidad y plazo combinables y probables por separado
State Machine	`domain/state_machine.py`	Impide estados imposibles (finalizar sin iniciar, resolver sin escalar)
Repository + Unit of Work + Outbox	`domain/repositories.py` , `infrastructure/`	Una transacción por caso de uso; eventos guardados atómicamente
Service Layer	`application/services.py`	Orquesta casos de uso sin lógica de negocio
Strategy	`RefundPolicy`	Nueva causa de reembolso = nueva clase (Open/Closed)
CQRS-light	`SqlDashboardReadModel`	El dashboard lee sin cargar aggregates

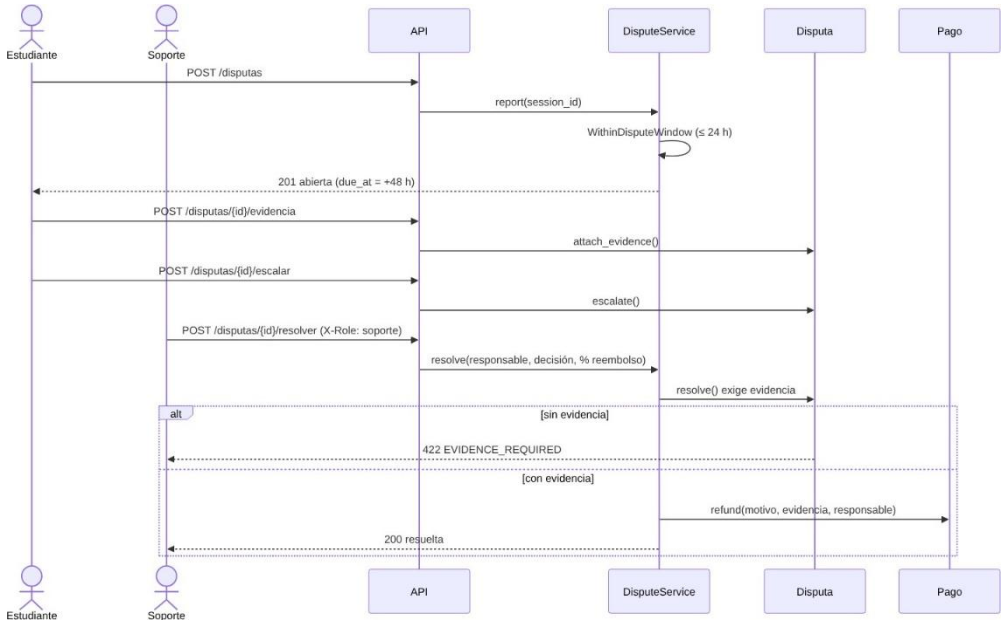
Adapter	`FakePaymentGateway`	La pasarela real se sustituye sin tocar el dominio

SOLID: responsabilidad única por aggregate; abierto/cerrado vía `Specification` y `RefundPolicy`; sustitución de repositorios (SQLite/PostgreSQL); puertos pequeños por aggregate; inversión de dependencias con raíz de composición única (`container.py`). Las **13 ADR** (7 heredadas de la Unidad 3 y 6 del PIA) están en `docs/architecture/decision-records.md`.

3.5 Secuencias principales



Reserva y pago



Disputa y resolución

4. Implementación

Elemento	Detalle
API REST	FastAPI; 35 rutas agrupadas en Tutores, Reservas, Sesiones, Disputas, Dashboard y Sistema

Elemento	Detalle
Errores	404 no existe · 409 estado/plazo/traslape · 422 regla de negocio · 401/403 rol
Persistencia	SQLAlchemy 2.0; tablas `estudiantes, tutores, reservas, sesiones, pagos, disputas, domain_events`
Dinero	`Decimal` con `ROUND_HALF_UP` ; importes como texto con 2 decimales
Roles	Cabecera `X-Role` (soporte, administrador, estudiante, tutor)

Asignación	Si la reserva no indica tutor, se asigna uno activo, disponible, con la modalidad y sin traslape
Notificaciones	`GET /notificaciones`: pago, ventana de disputa y SLA de Soporte por vencer

Funcionalidades adicionales respecto al prototipo de la Unidad 3: extensión de sesión con cargo proporcional (BR-0012), cambio de modalidad acordado (BR-0013), corte de conexión (BR-0010), reporte de comentarios (BR-0016), reactivación de tutores, evidencia obligatoria en reembolsos y SLA de disputas.

Contrato OpenAPI (Swagger UI disponible en /docs al ejecutar la app)

```
OpenAPI 3.1.0 — AulaPlus API v1.0.0  (35 rutas, generado desde FastAPI)

GET    /health                                  [Sistema] Health
GET    /notificaciones                          [Sistema] Avisos de plazos por vencer
POST   /tutores                                 [Tutores] Register Tutor
POST   /estudiantes                             [Tutores] Register Student
PUT    /tutores/{tutor_id}/disponibilidad       [Tutores] Availability
GET    /tutores/{tutor_id}/perfil               [Tutores] Profile
GET    /tutores/{tutor_id}/estado               [Tutores] Tutor Status
POST   /tutores/{tutor_id}/reactivar            [Tutores] Reactivate
POST   /reservas                                [Reservas] Reserve
GET    /reservas/{reserva_id}                   [Reservas] Get Reserva
POST   /reservas/{reserva_id}/pago              [Reservas] Pay
POST   /reservas/liberar-vencidas               [Reservas] Release
GET    /sesiones/{sid}                          [Sesiones] Get Session
POST   /sesiones/{sid}/presencia                [Sesiones] Confirmar presencia (BR-0008)
POST   /sesiones/{sid}/iniciar                  [Sesiones] Start
POST   /sesiones/{sid}/checklist                [Sesiones] Checklist
POST   /sesiones/{sid}/finalizar                [Sesiones] Finish
POST   /sesiones/{sid}/corte-conexion           [Sesiones] Falla de conexión (BR-0010)
POST   /sesiones/{sid}/cancelar                 [Sesiones] Cancel
POST   /sesiones/{sid}/ausencia-tutor           [Sesiones] Tutor Absence
POST   /sesiones/{sid}/ausencia-estudiante      [Sesiones] Student Absence
POST   /sesiones/{sid}/extension                [Sesiones] Solicitar extensión (BR-0012)
POST   /sesiones/{sid}/extension/aceptar        [Sesiones] Accept Extension
POST   /sesiones/{sid}/modalidad                [Sesiones] Solicitar cambio de modalidad (BR-0013)
POST   /sesiones/{sid}/modalidad/aceptar        [Sesiones] Accept Modality
POST   /sesiones/{sid}/calificaciones           [Sesiones] Rate
POST   /sesiones/{sid}/comentarios/reportar     [Sesiones] Reportar comentario (BR-0016)
POST   /sesiones/cerrar-vencidas                [Sesiones] Close Overdue
POST   /disputas                                [Disputas] Report
GET    /disputas/vencidas                       [Disputas] Disputas fuera del SLA de 48 h (BR-0011)
GET    /disputas/{did}                          [Disputas] Get Dispute
POST   /disputas/{did}/evidencia                [Disputas] Evidence
POST   /disputas/{did}/escalar                  [Disputas] Escalate
POST   /disputas/{did}/resolver                 [Disputas] Resolver disputa (solo Soporte; exige evidencia)
GET    /dashboard/semanal                       [Dashboard] Weekly
```

Contrato OpenAPI generado por la aplicación



Demo contra la API en ejecución

5. Calidad, CI/CD y despliegue

5.1 Pruebas

101 pruebas: unitarias de dominio (Specification, State Machine, aggregates) e integración HTTP con SQLite y PostgreSQL 16 (reservas, pagos, sesiones, disputas, dashboard, roles, persistencia/outbox, actualización de esquema y datos del laboratorio `08_AulaPlus`). Las pruebas verifican también que el OpenAPI versionado coincida exactamente con el contrato generado. (`tests/test_traceability.py`) falla si la matriz de trazabilidad cita una prueba inexistente o deja una regla sin cubrir. El resultado más reciente se documenta en `docs/evidence/Final_Validation.md`.

Las capturas 01–03 se conservaron del paquete fuente como evidencia histórica (90 pruebas). Para el paquete fusionado, la validación reproducible de 101 pruebas se registra en `docs/evidence/Final_Validation.md`.

Terminal — pruebas unitarias e integración con cobertura

```
$ pytest --cov=src --cov-fail-under=80
src/api/main.py                          141      1    99%
src/api/schemas.py                        72      0   100%
src/application/__init__.py                0      0   100%
src/application/ports.py                  16      0   100%
src/application/services.py              319      3    99%
src/domain/__init__.py                     0      0   100%
src/domain/disputa.py                     38      0   100%
src/domain/errors.py                      16      0   100%
src/domain/estudiante.py                   4      0   100%
src/domain/events.py                      15      0   100%
src/domain/pago.py                        58      0   100%
src/domain/repositories.py                13      0   100%
src/domain/reserva.py                     34      0   100%
src/domain/services.py                    31      0   100%
src/domain/sesion.py                     149      0   100%
src/domain/specifications.py              66      0   100%
src/domain/state_machine.py               21      0   100%
src/domain/tutor.py                       44      0   100%
src/domain/value_objects.py               90      0   100%
src/infrastructure/__init__.py             0      0   100%
src/infrastructure/config.py              15      0   100%
src/infrastructure/container.py           26      0   100%
src/infrastructure/database.py            15      1    93%
src/infrastructure/orm.py                 83      0   100%
src/infrastructure/payment_gateway.py      7      0   100%
src/infrastructure/read_model.py          25      0   100%
src/infrastructure/repositories.py       136      5    96%
src/infrastructure/unit_of_work.py        38      2    95%
-----------------------------------------------------------
TOTAL                                   1472     12    99%
Required test coverage of 80% reached. Total coverage: 99.18%
90 passed in 6.52s
```

Pruebas y cobertura (captura del paquete fuente)

Terminal — suite completa contra PostgreSQL 16

```
$ TEST_DATABASE_URL=postgresql+psycopg2://aulaplus:***@localhost:5432/aulaplus pytest --cov=src
src/infrastructure/database.py            15      1    93%
src/infrastructure/orm.py                 83      0   100%
src/infrastructure/payment_gateway.py      7      0   100%
src/infrastructure/read_model.py          25      0   100%
src/infrastructure/repositories.py       136      5    96%
src/infrastructure/unit_of_work.py        38      2    95%
-----------------------------------------------------------
TOTAL                                   1472     12    99%
Required test coverage of 80% reached. Total coverage: 99.18%
90 passed in 10.58s
```

Suite completa contra PostgreSQL 16



Análisis estático

5.2 Docker Compose, variables de entorno y CI/CD

- **Docker Compose** (`docker-compose.yml`): PostgreSQL 16 para desarrollo con volumen persistente, API FastAPI y un servicio PostgreSQL separado y efímero para `tests` (perfil `tests`). Ejecutar: `docker compose up --build` y `docker compose --profile tests run --rm tests`.
- **12-factor:** `TABASE_URL` `APP_ENV` `PORT`, credenciales de la base por entorno; `.env.example` sin secretos.
- **GitHub Actions** (`.github/workflows/ci.yml`): workflow configurado con `test` (ruff + pytest contra PostgreSQL), `docker` (build + pruebas en Compose)

y `deploy` en `main`, que requiere configurar `RENDER_DEPLOY_HOOK`.

- **Despliegue:** `La aplicación está desplegada en Render en https://pia-ndd7.onrender.com. La ejecución CI/CD #9 confirmó el deploy y que /health y /docs responden HTTP 200; Swagger/OpenAPI está disponible en /docs.`

5.3 Evidencia de CI/CD y despliegue

Verificación realizada el 6 de noviembre de 2026 en el repositorio [jyrnso0/PIA](https://github.com/jyrnso0/PIA),

Los tres jobs pasaron:

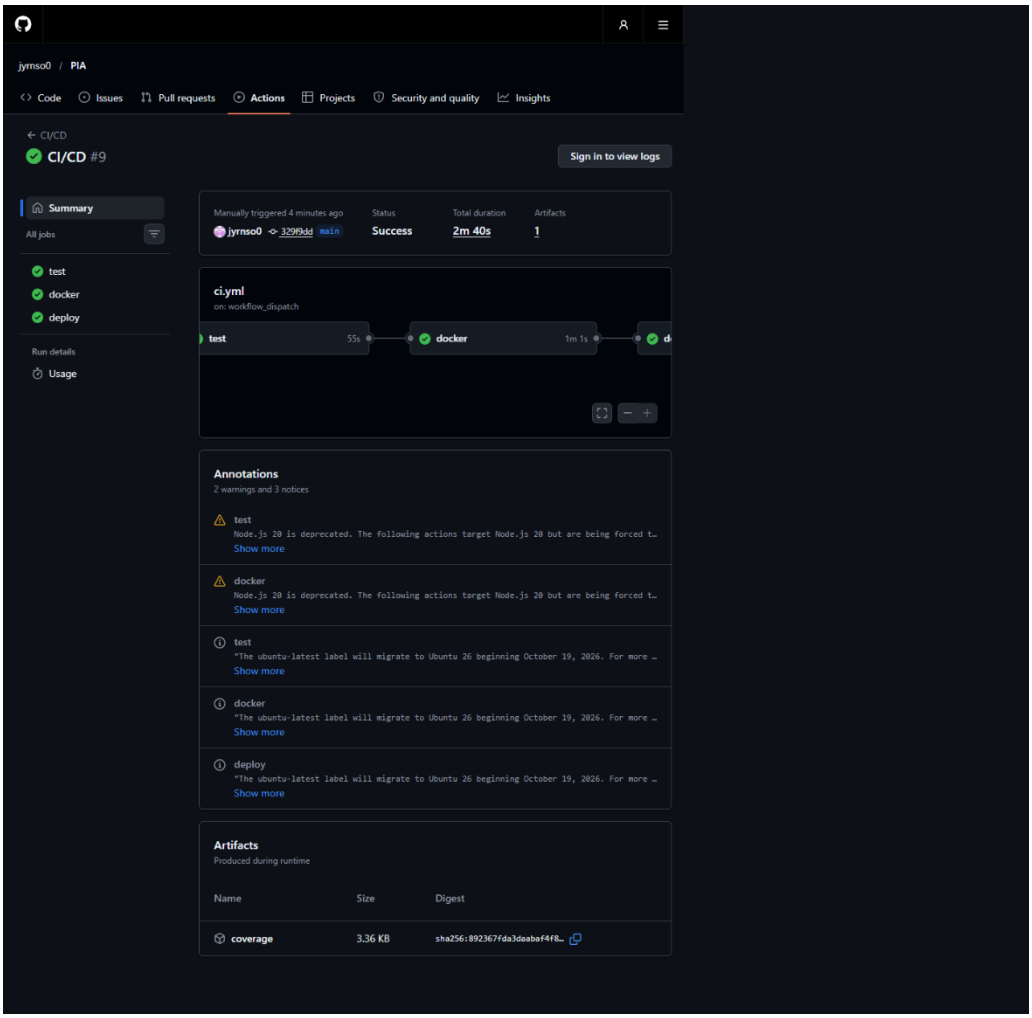
| Job | Resultado verificado |

| `test` | Ruff y pytest con PostgreSQL 16 finalizaron correctamente. |

| `docker` | Se construyó la aplicación y pasaron las pruebas de Docker Compose. |

| `deploy` | Se aceptó el deploy hook de Render y las verificaciones posteriores de `/health` y `/docs` finalizaron correctamente. |

La URL pública comprobada es <https://pia-ndd7.onrender.com>. En una comprobación directa el 6 de noviembre de 2026, `/health` respondió HTTP 200 con `{"status":"ok"}` y `/docs` respondió HTTP 200. La ejecución de Actions volvió a verificar ambos endpoints después de solicitar el despliegue.



6. Trazabilidad análisis → arquitectura → código → pruebas

Matriz completa en `docs/traceability.md` y `docs/traceability.csv` (regla → necesidad → historia → paso BPMN
→ dominio → API → pruebas).

Regla	Historia	Paso BPMN	Dominio	Pruebas
BR-0001	US-01	Registrar reserva → Pagar → Validar pago y reserva	Reserva (PaymentDeadline, State Machine)	4
BR-0002	US-02	Calificar sesión y tutor → Fin	Sesion.close / rate (State Machine REALIZADA→CERRADA)	4
BR-0003	US-03	Cancelar sesión (flujo alterno)	Sesion.cancel + ReportedAbsence (Specification)	2
BR-0004	US-04	Cancelar sesión → Calcular penalización	LateCancellation (Specification), Pago.penalty_for, RefundCalculator	6
BR-0005	US-05	Validar pago y reserva (desglose)	Pago.confirm → PaymentBreakdown	3
BR-0006	US-06	Asignar tutor según disponibilidad	OverlapValidator + NoOverlap (Specification)	6
BR-0007	US-07	Reportar disputa	WithinDisputeWindow (Specification), Disputa	3
BR-0008	US-08	Registrar asistencia y estado	Sesion.confirm_presence / mark_student_absent	3
BR-0009	US-09	Registrar asistencia y estado	Sesion.mark_tutor_absent, RefundCalculator, Tutor.register_absence	3
BR-0010	US-10	Realizar sesión	ConnectionCounts (Specification), Sesion.connection_cut	5

BR-0011	US-11	Brindar soporte → Cerrar ticket	Disputa.escalate / overdue (SLA)	2
BR-0012	US-12	Realizar sesión	Sesion.propose/accept_extension, Pago.add_extra	5
BR-0013	US-13	Tutor revisa y acepta	Sesion.propose/accept_modality_change	5
BR-0014	US-14	Realizar sesión → Registrar sesión y estado	Sesion.confirm_checklist / finish	5
BR-0015	US-15	Asignar tutor según disponibilidad	Tutor.set_availability, SlotWithinAvailability (Specification)	5
BR-0016	US-16	Calificar sesión y tutor	Tutor.add_rating, Sesion.report_comment	3
BR-0017	US-17	Brindar soporte → Cerrar ticket	Pago.refund (motivo, responsable, fecha)	3

Regla	Historia	Paso BPMN	Dominio	Pruebas
BR-0018	US-18	Registrar asistencia y estado	Tutor.register_absence / reactivate, TutorActive (Specification)	4
BR-0019	US-19	Consulta gerencial (read model)	SqlDashboardReadModel (CQRS-light)	1
BR-0020	US-20	Brindar soporte → Cerrar ticket	Disputa.resolve → EvidenceRequired	3

Requerimiento no funcional	Implementación	Pruebas
RNF-1 Trazabilidad de reembolsos	Pago.refund_responsible / refunded_at	2
RNF-2 Notificaciones de plazos	NotificationService · GET /notificaciones	3
RNF-3 Control de acceso por rol	require(Role) en la API (X-Role)	3
RNF-4 Consistencia en cálculos	Money (Decimal, ROUND_HALF_UP)	2
RNF-5 Persistencia / disponibilidad	SQLAlchemy + PostgreSQL, Unit of Work + outbox	3

Conclusiones:

La aplicación de los principios de Diseño Orientado al Dominio (DDD) y arquitectura hexagonal en AulaPlus permitió organizar la solución alrededor de sus reglas de negocio. La separación entre dominio, aplicación, API e infraestructura ayuda a mantener la lógica central independiente de tecnologías como FastAPI y SQLAlchemy, y facilita la evolución y el mantenimiento del sistema.

La implementación de los patrones Specification y State Machine permitió representar y validar reglas como los plazos de pago y disputa, las penalizaciones por cancelación y las transiciones válidas de reservas, sesiones, pagos y disputas. Además, el uso de objetos de valor y cálculos monetarios con Decimal contribuye a aplicar las reglas de manera consistente y a reducir errores en los importes.

Finalmente, la trazabilidad entre requerimientos, historias de usuario, arquitectura, código y pruebas permitió verificar la cobertura de las 20 reglas de negocio. Las 101 pruebas automatizadas y la validación con SQLite y PostgreSQL respaldan el funcionamiento del sistema en ambos motores.